

# PX4 全矢量（全驱动）六旋翼控制系统技术手册与源码实现参考

本手册为 **全矢量（全驱动）六旋翼控制系统** 的底层技术实现手册，详细阐述从机架门禁识别、位置/姿态解耦算法，到执行器控制分配的全链路源码设计与实现细节。

## 目录

- 一、机架安全白名单 (systemlib)
  - 1. src/lib/systemlib/fully\_actuated\_airframe.h
- 二、位置控制层 (mc\_pos\_control) 源码实现
  - 1. PositionControl/ControlMath.hpp / .cpp
  - 2. PositionControl/PositionControl.hpp / .cpp
  - 3. FullyActuatedControl.hpp / .cpp
  - 4. MulticopterPositionControl.hpp / .cpp (主集成)
  - 5. multicopter\_position\_control\_params.c
- 三、手动姿态控制层 (mc\_att\_control) 源码实现
  - 1. FullyActuatedControlMath.hpp
  - 2. FullyActuatedAttitudeControl.hpp / .cpp
  - 3. mc\_att\_control\_main.cpp (主集成)
  - 4. mc\_att\_control\_params.c
- 四、控制分配与混控层 (control\_allocator) 源码实现
  - 1. ControlAllocationPseudoinverseFullyActuated.hpp / .cpp
  - 2. ActuatorEffectivenessRotors.cpp (硬编码推力轴)
  - 3. FullyActuatedControlAllocation.hpp / .cpp 与 ControlAllocator.cpp
- 五、状态机协同与地面安全 (Commander)
  - 1. Commander.cpp (地面滑行动态防自动加锁)
- 六、机架启动脚本与仿真配置
- 七、核心参数字典与调试速查表

## 一、机架安全白名单 (systemlib)

### 1. src/lib/systemlib/fully\_actuated\_airframe.h

**文件路径:** src/lib/systemlib/fully\_actuated\_airframe.h

**功能职责:** 提供 `constexpr` 机架白名单判定，确保非全驱动机型绝不触发任何全驱动分支，零额外运行时开销。

CPP

```
namespace px4 {  
constexpr bool isFullyActuatedAirframe(const int32_t sys_autostart)  
{  
    // 6003: 真实全驱动六旋翼机架  
    // 4026: Gazebo SITL 仿真全驱动六旋翼机架  
    return (sys_autostart == 6003) || (sys_autostart == 4026);  
}  
}
```

## 二、位置控制层（mc\_pos\_control）源码实现

### 1. PositionControl/ControlMath.hpp / .cpp

**核心函数：** ControlMath::thrustNedToBody()

**数学原理：** 将位置环输出的地面 NED 期望推力  $F_{NED} = [F_N, F_E, F_D]^T$  转换到机体坐标系（FRD） $F_{body} = [F_x, F_y, F_z]^T$ 。

CPP

```
bool thrustNedToBody(const Vector3f &thr_sp_ned, const Quatf &q_current,
                    const Quatf &q_desired, vehicle_attitude_setpoint_s &att_sp)
{
    // 1. 合法性检查与四元数归一化
    Quatf q_current_normalized{q_current};
    q_current_normalized.normalize();
    Quatf q_desired_normalized{q_desired};
    q_desired_normalized.normalize();

    // 2. 核心坐标变换：必须使用当前物理姿态 q_current 进行逆旋转
    const Vector3f thrust_body = q_current_normalized.rotateVectorInverse(thr_sp_ned);

    // 3. 赋值目标姿态与机体三维推力
    q_desired_normalized.copyTo(att_sp.q_d);
    thrust_body.copyTo(att_sp.thrust_body);
    return true;
}
```

**设计依据：**必须使用当前物理姿态  $q_{current}$  而非期望姿态  $q_{desired}$  进行推力旋转。在倾斜与动态机动过程中，只有按实际当前物理姿态分解推力，空间中合成的合力方向才时刻指向目标方向，消除力—姿态动态耦合震荡。

## 2. PositionControl/PositionControl.hpp / .cpp

### 新增接口与核心改动：

1. `setDirectThrustControl(bool enabled)`：开启/关闭三维直接推力；
2. `setDirectThrustLimits(bool enabled, float hor_limit, float ratio)`：设置水平力绝对上限及与  $F_z$  的比率上限；
3. `_accelerationControlDirect()`：三维推力直接映射函数。

**核心代码实现：**

CPP

```
void PositionControl::_accelerationControlDirect()
{
    // 将三维加速度通过悬停推重系数线性映射为 NED 推力
    const float acceleration_to_thrust = _hover_thrust / CONSTANTS_ONE_G;
    _thr_sp.xy() = _acc_sp.xy() * acceleration_to_thrust;
    _thr_sp(2) = _acc_sp(2) * acceleration_to_thrust - _hover_thrust;
    _thr_sp(2) = math::min(_thr_sp(2), -_lim_thr_min);
}
```

**在 `_velocityControl(dt)` 中的力锥限制与 Anti-Windup:**

CPP

```
if (_direct_thrust_control && _direct_thrust_limits_enabled) {
    const Vector2f direct_thrust_sp_xy(_thr_sp);
    const float direct_thrust_sp_xy_norm = direct_thrust_sp_xy.norm();
    // 水平推力上限受绝对阈值与垂直推力力锥共同约束
    const float direct_thrust_max_xy = math::min(_lim_thr_xy_direct,
                                                _lim_thr_xy_to_z_ratio * fabsf(_thr_sp(2)));

    if ((direct_thrust_sp_xy_norm > direct_thrust_max_xy) && (direct_thrust_sp_xy_norm > FLT_EPSILON)) {
        _thr_sp.xy() = direct_thrust_sp_xy * (direct_thrust_max_xy / direct_thrust_sp_xy_norm);
    }
}

// 用限幅后的推力反算实际加速度，供速度积分器进行抗饱和 (Tracking Anti-Windup)
const Vector2f acc_sp_xy_produced = Vector2f(_thr_sp) * (CONSTANTS_ONE_G / _hover_thrust);
```

**3. `FullyActuatedControl.hpp / .cpp`****核心类：** `FullyActuatedControl`**功能职责：** 管理位置环内的全驱动状态机、遥控器摇杆整形、Pose 模式定点锁位及姿态生成。

核心成员与函数：

| 函数 / 成员                                     | 代码逻辑与功能说明                                                                                                                                                                                                                                                                                 |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>updateSubscriptions()</code>          | 检查机架白名单（赋值 <code>_fullyactuated_enabled</code> ）；读取 <code>vehicle_attitude</code> ，并在 <code>quat_reset_counter</code> 变化时清空姿态滤波器记忆；读取 <code>manual_control_setpoint</code> 。                                                                                                              |
| <code>updateControlMode()</code>            | 识别 Altitude / Position 上下文；通过 <code>resolveMode()</code> 解析 Mode 0 或 Mode 1；在模式切换时触发 <code>reset()</code> 。                                                                                                                                                                               |
| <code>updateSetpoint()</code>               | 在 Mode 1（Pose 模式）且起飞在空中时，将无人机当前位置锁存为 <code>_position_hold</code> 并覆盖 <code>_setpoint.position</code> ，速度置 0；输出 <code>DirectThrustConfiguration</code> 配置。                                                                                                                                 |
| <code>generateAttitudeSetpoint()</code>     | <b>Mode 0</b> ：生成水平姿态 $q_d = \text{Euler}(0, 0, Yaw_{sp})$ ；<br><b>Mode 1</b> ：摇杆经过 <code>math::expo_deadzone</code> 、低通滤波 <code>_tilt_filter</code> 及 <code>MPC_FA_TILT_MAX</code> 限幅，合成期望姿态 $q_d = q_{yaw} \times q_{tilt}$ ；<br>调用 <code>ControlMath::thrustNedToBody</code> 将推力转入机体系。 |
| <code>adjustPositionHoldForEKReset()</code> | 当 EKF 发生坐标重置跳变时，同步将 <code>delta_xy</code> / <code>delta_z</code> 补偿到内部定点坐标 <code>_position_hold</code> 。                                                                                                                                                                                  |

4. `MulticopterPositionControl.hpp / .cpp`（主集成）

主控制循环 `Run()` 中的接入点：

1. 输入更新（L405）：`_fully_actuated_control.updateSubscriptions();`
2. 模式使能复位（L413, L418）：位置控制模式切入/切出时调用 `_fully_actuated_control.reset();`
3. 模式更新（L423）：`_fully_actuated_control.updateControlMode(_vehicle_control_mode);`
4. 推力配置下发（L532 ~ L536）：

CPP

```
const FullyActuatedControl::DirectThrustConfiguration direct_thrust =
    _fully_actuated_control.updateSetpoint(states.position, flying, flying_but_ground_contact, _setp
oint);
_control.setDirectThrustControl(direct_thrust.enabled);
_control.setDirectThrustLimits(direct_thrust.limits_enabled, direct_thrust.horizontal_limit,
    direct_thrust.horizontal_to_vertical_ratio);
```

## 5. 姿态与推力发布 (L627 ~ L631) :

CPP

```
if (!_fully_actuated_control.generateAttitudeSetpoint(local_pos_sp, dt, attitude_setpoint)) {
    // 安全兜底：降级回传统倾角映射
    _control.getAttitudeSetpoint(attitude_setpoint);
}
attitude_setpoint.timestamp = hrt_absolute_time();
_vehicle_attitude_setpoint_pub.publish(attitude_setpoint);
```

## 6. EKF 重置对齐 (L712) : 在 `adjustSetpointForEKFResets()` 中调用

`_fully_actuated_control.adjustPositionHoldForEKFRreset(...)`。

## 5. `multicopter_position_control_params.c`

注册位置环新增参数：

- `MPC_FA_MODE` : `0` = 水平全驱动（锁平，三维直接推力改位置），`1` = Pose 模式（定点锁 XYZ 坐标，摇杆控制机身目标姿态）；
- `MPC_FA_RC_AUX` : `0` = 禁用，`1~6` 对应 AUX1~AUX6；
- `MPC_FA_TILT_MAX` : Pose 模式最大倾角（默认 15.0°）；
- `MPC_FA_XY_THR` : 最大水平推力上限（默认 0.15）；
- `MPC_FA_XY_RATIO` : 力锥比率上限（默认 0.30）。

## 三、手动姿态控制层 (mc\_att\_control) 源码实现

### 1. `FullyActuatedControlMath.hpp`

纯函数数学库：

- `selectThreePositionMode(aux, prev_mode, valid)` : 将三段开关映射为 0/1/2 模式（低位  $\leq -0.6 \rightarrow 0$ ，中位  $[-0.2, 0.2] \rightarrow 1$ ，高位  $\geq 0.6 \rightarrow 2$ ），区间带有施密特滞回。
- `updateAttitudeModeState()` : 模式 2（地面滑行）**仅允许在已着陆（`landed`）状态下切入**；空中请求模式 2 自动

CPP

```

inline GroundTaxiSetpoint groundTaxiSetpoint(Vector2f normalized_stick,
      float xy_limit_absolute, float xy_to_z_ratio, float z_limit)
{
    GroundTaxiSetpoint setpoint{};
    // 1. 水平力限幅
    setpoint.xy_limit = math::min(xy_limit_absolute, xy_to_z_ratio * z_limit);
    const Vector2f thrust_xy = normalized_stick * setpoint.xy_limit;
    const float thrust_xy_norm = thrust_xy.norm();
    setpoint.thrust_body(0) = thrust_xy(0);
    setpoint.thrust_body(1) = thrust_xy(1);

    // 2. 根据单向旋翼推力锥，自动生成最小垂直推力（使旋翼产生水平力时不离地）
    if (thrust_xy_norm > FLT_EPSILON) {
        setpoint.thrust_body(2) = -math::min(z_limit, thrust_xy_norm / xy_to_z_ratio);
    }
    return setpoint;
}

```

## 2. FullyActuatedAttitudeControl.hpp / .cpp

核心类：FullyActuatedAttitudeControl

功能职责：管理 Stabilized 下的全驱动状态机、遥控器摇杆滤波与推力生成。

- `updateMode()`：根据 `MPC_FA_STAB` / `MPC_FA_STAB_AUX`、当前是否 Stabilized 模式、着陆状态在模块内部计算当前有效模式。
- `generateAttitudeSetpoint()`：
  - **模式 1（空中锁平）**：机身保持水平  $q_{sp} = \text{Euler}(0, 0, Yaw_{sp})$ ，摇杆映射水平推力并转为机体系 `thrust_body`；
  - **模式 2（地面滑行）**：机身姿态跟随当前地面倾角  $q_{sp} = \text{Euler}(Roll_{current}, Pitch_{current}, Yaw_{sp})$ ，推力由 `groundTaxiSetpoint` 算出。

## 3. mc\_att\_control\_main.cpp（主集成）

- 构造函数中实例化 `_fully_actuated_attitude_control(this)`；
- 在主循环中更新模式并判断切换：

CPP

```
if (_fully_actuated_attitude_control.updateMode(_manual_control_setpoint,
    stabilized_manual_control, _landed, _vtol)) {
    _man_roll_input_filter.reset(0.f);
    _man_pitch_input_filter.reset(0.f);
}
```

- 在 `generate_attitude_setpoint()` 中优先调用全驱动解算，失败则走传统角度映射。

#### 4. `mc_att_control_params.c`

注册姿态环新增参数：

- `MPC_FA_STAB`：0 = 传统倾角自稳，1 = 空中锁平直推，2 = 地面滑行；
- `MPC_FA_STAB_AUX`：1~6 对应三段开关；
- `MPC_FA_GND_XY_Z`：地面滑行力锥比率（默认 0.30）；
- `MPC_FA_GND_ZMAX`：地面滑行最大向上推力（默认 0.20）。

## 四、控制分配与混控层（`control_allocator`）源码实现

### 1. `ControlAllocationPseudoInverseFullyActuated.hpp / .cpp`

**核心算法：**三维推力统一尺度伪逆分配（Uniform Thrust Scale）。

CPP

```
void ControlAllocationPseudoInverseFullyActuated::updateThrustScale()
{
    float thrust_scale = 1.f;
    // 优先采用 z 轴的分配尺度，若无则使用其他有效轴
    for (int axis_idx = 2; axis_idx >= 0; --axis_idx) {
        int num_non_zero_thrust = 0;
        float norm_sum = 0.f;
        for (int i = 0; i < _num_actuators; i++) {
            const float norm = fabsf(_mix(i, THRUST_X + axis_idx));
            norm_sum += norm;
            if (norm > FLT_EPSILON) { ++num_non_zero_thrust; }
        }
        if (num_non_zero_thrust > 0) {
            thrust_scale = norm_sum / num_non_zero_thrust;
            break;
        }
    }
    // Fx, Fy, Fz 统一使用相同的缩放尺度，保持空间合力方向不失真
    _control_allocation_scale(THRUST_X) = thrust_scale;
    _control_allocation_scale(THRUST_Y) = thrust_scale;
    _control_allocation_scale(THRUST_Z) = thrust_scale;
}
```

## 2. ActuatorEffectivenessRotors.cpp（硬编码推力轴）

配置代号：CA\_AIRFRAME = 16（FixedFullyActuatedHexa）

为了量产和防误触，将 6 个电机的空间几何推力轴向量硬编码在固件内部（FRD 机体系）：

CPP

```
static constexpr float k_axes[6][3] = {
    {-0.519837f, -0.422618f, -0.742404f}, // M1 右前 CW
    {-0.519837f,  0.422618f, -0.742404f}, // M2 左前 CCW
    {-0.106080f,  0.661501f, -0.742404f}, // M3 正左 CW
    { 0.625917f,  0.238883f, -0.742404f}, // M4 左后 CCW
    {-0.106080f, -0.661501f, -0.742404f}, // M5 正右 CCW
    { 0.625917f, -0.238883f, -0.742404f}, // M6 右后 CW
};
```

在 QGC 地面站中选择机架类型为 16 时，UI 将不再展示电机的 Axis X/Y/Z 参数输入框，仅保留位置（PX/PY/PZ）和转向（KM）。

### 3. FullyActuatedControlAllocation.hpp / .cpp 与 ControlAllocator.cpp

- `createPseudoInverse()`：对于白名单机架自动实例化 `ControlAllocationPseudoInverseFullyActuated`；
- `createEffectivenessSource()`：针对 `CA_AIRFRAME=16` 实例化绑定 `FixedFullyActuatedHexa` 的多旋翼混控模型。

## 五、状态机协同与地面安全（Commander）

### 1. Commander.cpp（地面滑行动态防自动加锁）

**核心需求：**常规多旋翼在着陆后会触发 `COM_DISARM_LAND`（默认 2 秒自动加锁停机）。但在\*\*地面滑行模式（Ground Taxi）\*\*下，无人机需要像地面移动小车一样持续滑行操控。

**实现逻辑：**

- `mc_att_control` 在激活地面滑行模式时发布极简状态 `fully_actuated_control_status`（仅含 `ground_taxi_active` 与新鲜时间戳）；
- `Commander::handleAutoDisarm()` 实时检查该状态：若处于地面滑行且着陆在地面，则主动重置 `_auto_disarm_landed`

CPP

```
fully_actuated_control_status_s fa_status{};
const bool fa_ground_taxi = _fully_actuated_control_status_sub.copy(&fa_status)
    && fa_status.ground_taxi_active
    && (hrt_elapsed_time(&fa_status.timestamp) < 500_ms)
    && _vehicle_land_detected.landed;

if (fa_ground_taxi) {
    _auto_disarm_landed.set_state_and_update(false, hrt_absolute_time());
}
```

## 六、机架启动脚本与仿真配置

### 1. 机架启动脚本

- 真机启动脚本：ROMFS/px4fmu\_common/init.d/airframes/6003\_fully\_actuated\_hexa
- SITL 启动脚本：ROMFS/px4fmu\_common/init.d-posix/airframes/4026\_gz\_fully\_actuated\_hexa

SH

```
#!/bin/sh
. ${R}etc/init.d/rc.mc_defaults

param set-default SYS_AUTOSTART 4026
param set-default CA_AIRFRAME 16
param set-default CA_ROTOR_COUNT 6
param set-default CA_METHOD 0
param set-default MPC_FA_MODE 0
param set-default MPC_FA_TILT_MAX 15.0
param set-default MPC_FA_STAB 1
param set-default MPC_FA_XY_THR 0.15
param set-default MPC_FA_XY_RATIO 0.30
param set-default MC_YAW_TQ_CUTOFF 0.0
```

## 七、核心参数字典与调试速查表

| 参数名             | 默认值  | 范围         | 详细功能说明                                                                                                                                         |
|-----------------|------|------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| MPC_FA_MODE     | 0    | 0 ~ 1      | <b>位置控制全驱动模式：</b> <ul style="list-style-type: none"> <li>0：水平全驱动（锁平，三维直接推力改位置）</li> <li>1：Pose 模式（定点锁 XYZ 坐标，摇杆控制机身目标姿态）</li> </ul>            |
| MPC_FA_RC_AUX   | 0    | 0 ~ 6      | <b>位置模式 AUX 开关通道：</b> 0 禁用；1~6 对应 AUX1~AUX6（<-0.2 切 Mode 0，>0.2 切 Mode 1，带滞回）                                                                  |
| MPC_FA_TILT_MAX | 15.0 | 1.0 ~ 45.0 | <b>Pose 模式最大倾角（度）：</b> 遥控器满杆时允许倾斜的最大角度                                                                                                         |
| MPC_FA_STAB     | 0    | 0 ~ 2      | <b>自稳模式全驱动配置：</b> <ul style="list-style-type: none"> <li>0：标准倾角映射</li> <li>1：空中姿态锁平 + 摇杆直接映射水平力</li> <li>2：地面滑行模式（跟随地面倾角 + 最小向上力耦合）</li> </ul> |
| MPC_FA_STAB_AUX | 0    | 0 ~ 6      | <b>自稳模式 AUX 开关通道：</b> 1~6 对应三段开关（低位=0，中位=1，高位=2）                                                                                               |
| MPC_FA_XY_THR   | 0.15 | 0.0 ~ 1.0  | <b>水平推力绝对归一化上限：</b> 防止过大水平力导致电机饱和                                                                                                              |
| MPC_FA_XY_RATIO | 0.30 | 0.0 ~ 1.0  | <b>水平力与垂直力锥比率：</b> $ F_{xy}  \leq \text{RATIO} \times  F_z $                                                                                   |
| MPC_FA_GND_XY_Z | 0.30 | 0.0 ~ 1.0  | <b>地面滑行力锥比率：</b> 用于反算最小向上推力 $F_z = - F_{xy} /\text{RATIO}$                                                                                     |
| MPC_FA_GND_ZMAX | 0.20 | 0.0 ~ 1.0  | <b>地面滑行最大向上推力上限</b> （运行时另受 $0.8 \times F_{\text{hover}}$ 限制，确保不离地）                                                                             |
| CA_AIRFRAME     | 16   | -          | <b>执行器机架类型：</b> 16 对应固化推力轴全驱动六旋翼                                                                                                               |